

Report

人工智能导论—文本情感分类实验报告

车33 石亦烜 2023010302

⚠ Attention

代码链接见: <https://cloud.tsinghua.edu.cn/d/bcc1fd38dda94641bcec/>
使用方法见2.4 测试说明部分

1 模型分析

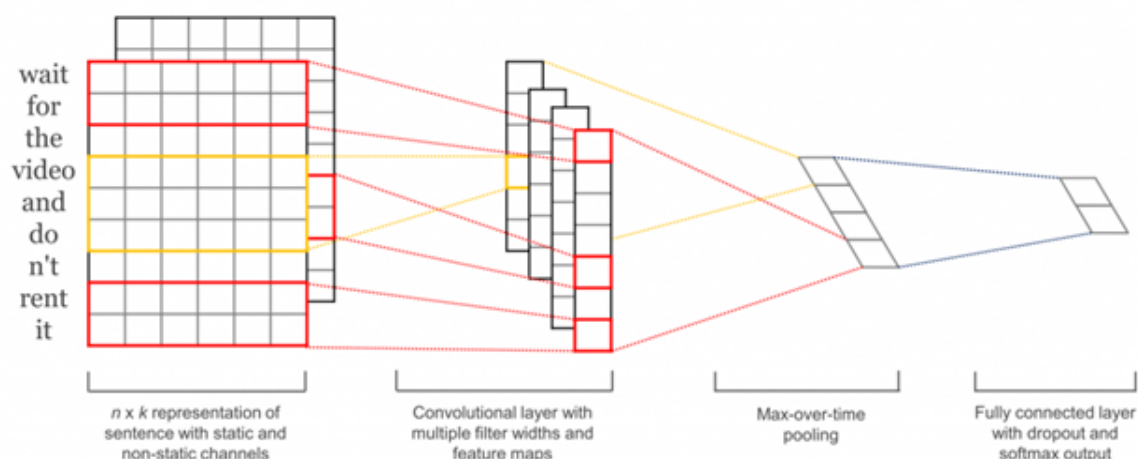
1.1 word2vec模型

每个句子已经做了分词, 假设一个句子有 n 个单词, 第 i ($1 \leq i \leq n$) 个单词记为 x_i , 利用给定的word2vec模型, 通过Word-Embedding方法将其表示为 k 维向量。因此一个句子可以表示为:

$$X_{i:n} = (x_1, x_2, \dots, x_n)^T \in \mathbb{R}^{n \times k}$$

实验中, 对每个batch中的句子填充到相同的长度。

1.2 CNN模型



一个从第 i 个单词开始包含 h 个词的窗口表示为

$$X_{i:i+h-1} = (x_i, x_{i+1}, \dots, x_{i+h-1})^T \in \mathbb{R}^{h \times k}$$

卷积核为宽度和词向量长度相同的矩阵：

$$W \in \mathbb{R}^{h \times k}$$

通过一个卷积核在窗口上完整扫描一遍，得到特征图：

$$C = (c_1, c_2, \dots, c_{n-h+1})^T$$

其中：

$$c_i = \text{ReLU}(WX_{n-h+1} + b)$$

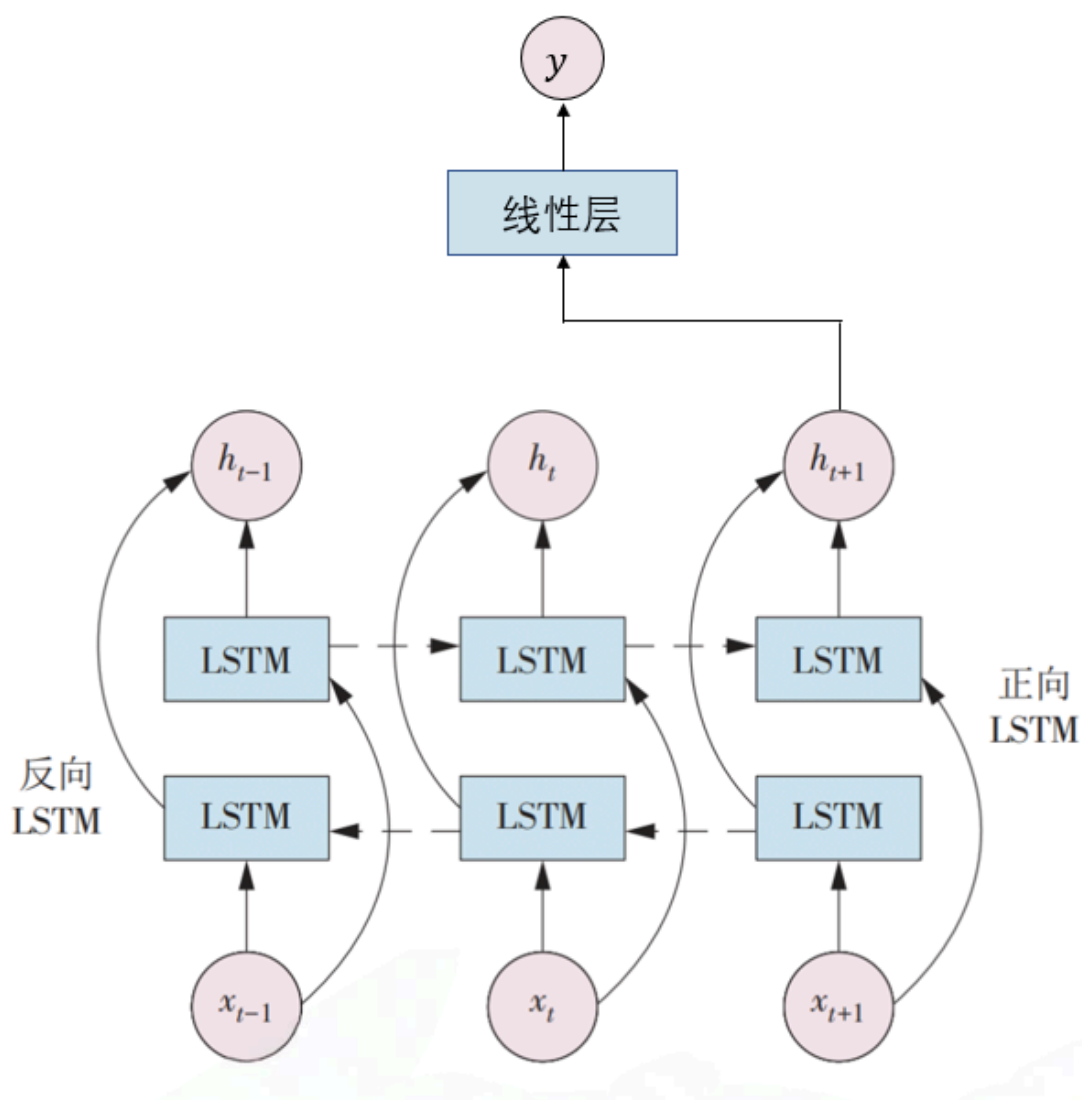
随后进行最大池化，得到

$$\hat{C} = (\hat{c}_1, \dots, \hat{c}_m)^T$$

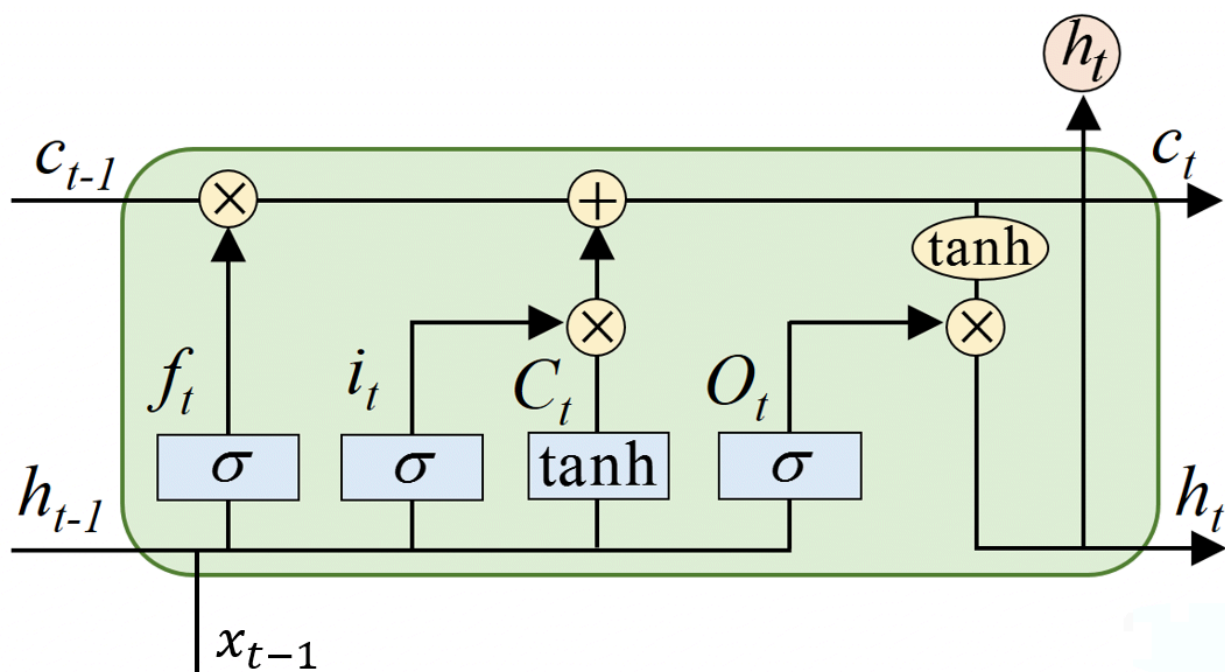
其中m为卷积核的个数

对 $\hat{C}$ 经过一次全连接层，得到最终的特征提取结果

1.3 Bi-LSTM模型



双向LSTM模型结构如图，其中LSTM模块如下：



1. 句子序列中的词向量按顺序作为LSTM模块的输入，同时每个LSTM模块中，输入上一时刻的隐藏值 h_{t-1} 和细胞状态(cell state) c_{t-1} ，经过遗忘门、输入门和输出门，输出当前时刻的隐藏值 h_t 和 c_t
2. 双向进行传播，取两个方向传播各自的最后一个隐藏状态进行拼接，作为线性层的输入
3. 线性层接收LSTM产生的两个隐藏状态的拼接，输出分类的预测结果 $y \in \mathbb{R}^2$

1.4 Attention-LSTM模型

为了更好地利用LSTM每个隐藏层的结果，引入Attention机制整合LSTM的输出结果，同时减轻梯度消失问题
也就是说，对LSTM的所有输出结果，有

$$c = f(output) = \sum_{i=1}^n a_i \cdot output_i$$

其中，

$$a = v^T \cdot \text{softmax}(W \cdot output_i)$$

其中， v, W 均为模型参数

1.5 MLP模型

使用MLP作为baseline

包含一个隐藏层和一个输出层。隐藏层接受一个已经完成词嵌入的batch，输出指定维数的张量。经过激活函数ReLU和Dropout。输出层接收隐藏层的结果，输出预测分类的向量

2 实验流程

2.1 数据加载

1. `datareader.py`：中实现了 `Mydata` 类，建立词汇-索引字典，同时实现了 `read_data` 方法，读取对应的数据集（训练集、验证集、测试集），将数据集中的句子根据词汇表转化为索引序列。同时实现了 `load_word2vec_embeddings` 方法，根据已有的word2vec模型将词汇表转化为词嵌入矩阵，以供 `torch.nn.Embedding` 使用
2. `dataset.py`：继承torch的Dataset类实现了 `Mydataset` 类，实现相应接口以供 DataLoader 使用

2.2 训练

1. 采用DataLoader动态加载数据集进行训练。损失函数采用交叉熵损失函数，优化器采用 `torch.optim.Adam` 动态调整学习率
2. 训练停止。采用早停机制，为防止过拟合，实验利用验证集测试每次epoch的性能（以F1分数）指标，记录下使得验证集效果最好的模型参数为最优参数。当连续10次epoch验证集的得分都没有优化时，终止训练，以记录的最优参数为训练结果。至多进行50个epoch
3. 训练参数：
 - 初始学习率：0.001
 - batch_size：64
 - 词嵌入维数：50
 - device：采用cuda加速

2.3 计算结果

在主函数中实现了 `evaluate_model` 方法。利用 `sklearn.metrics` 已有的方法计算准确率和f1-score

2.4 测试说明

通过添加命令行参数，实现了在运行时选择模型。本次共实现了 `CNN`, `RNN`, `LSTM`, `AttentionRNN`, `MLP` 五个模型

在命令行输入

SHELL

```
python main.py [--model MODEL]
```

即可运行，其中 **MODEL** 可选上述的五种模型。

输出时，会输出每次epoch过后验证集上的测试结果，以及是否更新最佳模型。最终输出使用最佳模型在测试集上的测试结果。

3 实验结果和对比

3.1 CNN, LSTM和MLP

经过调参，模型的最优性能如下表：

	CNN	Bi_LSTM	MLP
Accuracy	85.37%	85.09%	77.78%
F1_score	84.75%	85.01%	78.67%

1. **CNN和Bi_LSTM**：模型性能相近，准确率和F得分均在85%左右。

- 对于CNN，实验中尝试过增加卷积核种类及filter的数量，经调整测试后，发现选用高度为[3.4.5]的卷积核，每种卷积核数量为100得到了最优结果。具体可见参数对比部分。
- 而对于Bi_LSTM。最开始实现的是单向的LSTM，不管怎么调参F得分最高只有78%左右。改为双向LSTM后，模型性能有了较大的提升，说明双向扫描能够捕获更充分的上下文信息。

2. **MLP**：作为Baseline，MLP预料之中的是三者中性能最低的。上表中给出的MLP是只有一个隐藏层的，实验中我尝试过增加MLP的深度和增加隐藏层的神经元个数以求更高的性能，但并未成功。我推测这是因为本身训练数据集并不是特别大，过于复杂的MLP反而导致了模型参数量过大，过拟合更加严重。

3. 训练速度方面，CNN最快，大约在第10个epoch在验证集上取得了最优结果，且Loss收敛最快。Bi_LSTM居中，第18轮取得最优模型。而MLP直到第28轮才取得最优模型

3.2 关于几个RNN模型的对比

在此次实验的过程中我一共实现了几种不同方式的RNN，

第一种是朴素的RNN，即单向传播，不引入其余门。

第二种是用Attention机制修复的RNN，单向传播，原理前面已述。

第三种是在朴素RNN的基础上，最后输入线性层之前增加一个最大池化层以捕获整个序列的信息，同样为单向传播，即

$$c = f(output, hidden) = \max(output)$$

我们把它记作Pooling_RNN

第四种即前面的Bi_LSTM

四种模型经调参后的最佳性能如下表

	朴素RNN	AttentionRNN	Pooling_RNN	Bi_LSTM
Accuracy	62.14%	82.03%	82.38%	85.09%
F1_score	60.21%	81.68%	83.79%	85.01%

1. **朴素RNN**：性能出乎意料的差，几乎比随机预测结果没有优化多少。检查训练过程，发现朴素RNN的Loss下降极慢，且出现突然回弹现象。因此我推测出现了梯度消失问题，导致朴素RNN模型几乎没有性能。
2. **AtteneionRNN**：为了解决梯度消失，同时更好的学习每个隐藏值的结果，引入了Attention机制。结果表明Attention机制对梯度消失的现象起到了缓解作用，但性能并没有我预期的那么好。我推测是由于引入Attention后参数量过大的原因。
3. **Pooling_RNN**：这是比较出乎意料的，仅仅加了一个池化层，朴素RNN的性能就得到了极大的优化。经过与同学讨论，推测是池化层的加入规避了某些极端数据的影响。
4. **Bi_LSTM**：前面已讨论过。说明双向传播对模型学习上下文语义和缓解RNN的梯度消失问题是起到了很大作用的。

4 参数对比

4.1 初始学习率

这里选用CNN，调整Adam优化器的初始学习率，结果如下

初始学习率	0.0005	0.001	0.0015	0.002
Accuracy	82.14%	85.37%	82.38%	80.09%
F1_score	81.76%	84.75%	83.79%	80.31%

说明初始学习率0.001就是一个良好的参数。学习率过小，模型收敛速度慢；学习率过大，模型容易局部收敛或者反而跨过最小值

4.2 RNN的num_layers

这是让我印象比较深的一个参数。表示RNN的层数，若大于1，则序列经过一层RNN后，会将前一层的最后一个隐藏状态再输入下一层，再次进行学习。

一开始我觉得这个层数适当设置多一些，能够有助于RNN学到更好的性能，但实际调参后的结果如下（以Bi_LSTM为例）：

num_layers	1	2	3
Accuracy	85.09%	81.51%	82.11%
F1_score	85.01%	82.10%	80.79%

结果和我预料的恰恰相反。增加RNN的层数并没有使性能得到提高，相反，它不仅训练速度更慢，而且降低了模型性能。关于其原因，我推测有两点：

1. 训练数据过少：严格来说这次实验的训练集是不大的。因此提高RNN的层数就引入了更多参数，导致过拟合现象更加严重。
2. 序列较长：在观察数据集的时候我发现，某些句子长度很高，这样一来如果DataLoader采用shuffle，且对每个batch都将句子序列padding到相同长度，就会导致一些批次训练数据都有很长的序列长度，层数一多，更容易导致梯度消失现象。

5 问题思考

5.1 训练停止时间

训练停止时间主要是两种：

1. 设定固定的迭代次数，按时停止。但采用这种方式，就用不上给定的验证集，对epoch_num参数设置的要求就很高。如果设定过低，模型可能还没收敛；如果设定的过高，模型可能已经过拟合了，同样没有很好的性能。而且，实际实验下来发现不同模型的收敛速度是有差异的，意味着很难用一个统一的epoch_num参数来训练所有模型。
2. 利用验证集实现早停(Early Stopping)。设置一个迭代次数的上限，和早停参数(patience)迭代过程中记录当前的最优模型，每次迭代后模型在验证集上的表现与之相比来判断是否停止，如果连续几次(patience)在验证集上的表现都没有优化，则触发早停。
实验中采用的是第二种方法，迭代上限为50，patience=10。通过验证集实现早停机制，一定程度上可以防止模型产生过拟合，同时也避免测试集数据受到污染（即学习测试集）

5.2 参数初始化

一开始我对参数初始化的问题是有疑问的，因为调用了torch.nn.module类，我似乎并没有对模型参数进行初始化，就有不错的训练速度。后面经过查询资料，我知道了是Pytorch已经给各个模型采用了高效的默认初始化。其中，CNN和Linear层的权重采用kaiming初始化，RNN的权重采用正态分布初始化，而权重均为零初始化。下面阐述一些初始化方法的特点：

1. Xavier 初始化

包括按均匀分布和正态分布初始化两种。也就是权值矩阵：

$$W \sim \mathcal{U} \left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}} \right) \text{ 或 } W \sim \mathcal{N} \left(0, \frac{2}{n_{in} + n_{out}} \right)$$

其中 n_{in} 和 n_{out} 分别表示输入和输出的神经元个数

这种初始化能让经过一层网络后输出和输入的方差保持某种关系，及传播过程中方差保持相对稳定。从而一定程度上避免梯度消失或者爆炸

但是由于ReLU激活函数会截断负值，所以这种初始化不适合用于ReLU激活函数时，更适合tanh和sigmoid激活函数。

2. kaiming初始化

它和Xavier初始化有相似之处，不同在于kaiming尤其适用于ReLU激活函数。而ReLU又是现在用的最广的激活函数，因此kaiming初始化也用的很广泛。其均匀分布和正态分布的形式如下：

$$W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}}}}, \sqrt{\frac{6}{n_{\text{in}}}}\right) \text{ 或 } W \sim \mathcal{N}\left(0, \frac{2}{n_{\text{in}}}\right)$$

3. 正交初始化

将权值矩阵初始化为正交矩阵，即

$$W^T W = I$$

这样使不同神经元间具有独立性，能够缓解梯度消失问题

4. 普通的均匀分布和正态分布初始化

除了按照kaiming和Xavier的方法，还可以简单的采用一般的均匀分布（即零均值）或者高斯分布来随机初始化权值矩阵。然而这样的问题在于对分布特征数的设定有较高的要求。

比如，采用正态分布 $W \sim \mathcal{N}(0, \sigma^2)$ 初始化，如果分布标准差设定的不合适，就可能导致梯度爆炸或消失。对均匀分布同理，范围的选择要准确才能有好的性能

5.3 过拟合问题

对于本次实验，我觉得有可能导致过拟合的原因有：

1. 训练集过小或数据普遍性不足。就本次实验而言，训练集句子共大约20000条句子，这个数据量是相对较小的，同时由于汉语的复杂性，能传达情感倾向的方式非常丰富，也导致这样样本量很难较为全面的覆盖到文本情感表达的各种可能。
2. 模型过于复杂，参数量太大。这一点在前面关于RNN和MLP的讨论中已经体现，增加模型的深度反而导致对训练样本的过拟合，特别是在训练集本身普遍性不足时，就会更加降低模型的泛化能力。
3. 训练迭代次数太多，导致对训练集数据过拟合。

解决的方式也就可以看出了：

1. 增大训练数据量
2. 适当简化模型，或适当增大dropout方法的dropout_rate，再丢弃一些神经元。
3. 利用验证集进行早停。
4. 正则化：即在损失函数中增加惩罚项，使模型在训练过程中同时减小参数量

5.4 CNN, RNN和MLP对比

在本次实验中三种模型的性能对比在3.1部分已经分析。下面总结三种模型的特点：

1. **CNN**：此次实验中表现最好的模型。一方面由于可以并行计算，训练速度最快。同时可提取特征。但对序列中上下文信息的提取可能不全面，会丢失语义
 2. **RNN**：此次任务中和CNN表现接近。擅长处理文本这样的序列信息，保留了更多的上下文时序信息，但也因此不能并行计算，速度较慢。同时容易产生梯度消失的问题
 3. **MLP**：最基础的模型，模型结构很简单，也能有及格的性能。但上限不高，可解释性和表达性很差。很容易导致参数量过大而过拟合。
-

6 心得体会

report写到这里，我也长出了一口气。此次实验的过程可以说充满了曲折，我也更加感到纸上谈兵，不如身体力行。深度学习绝不是课上听听就能学会的，真正自己实现模型完成任务时才发现处处是需要注意的细节，任何一个参数的微小改变都可能带来性能的天壤之别。让我印象深的有这几点：

1. 第一是对梯度消失的直观感受。在第一次实现朴素的RNN时，模型性能之差让我反复调参也无法挽回。检查了loss更新才发现应该是梯度消失，在采用最大池化和Attention方法优化后果然有了显著的改善。
2. 第二是过拟合。在每个模型一开始设置时，我都想当然地把参数都设置的尽量大。比如隐藏层维数，RNN的层数，卷积核数量等等，性能都不好。直到调参时才意识到，大道至简。
3. 接受失望和意料之外。训模型，有时就是带着一丝玄学的意味。自己以为会性能很好的模型，结果可能就是平平而已。在实现AttentionRNN时，我坚信它会取得很好的性能，最后的结果却还不如仅仅加了一层池化的RNN，这样的失望，可能就是需要接受的常态。

这次实验是我第一次完整地实现一个深度学习任务。从了解什么是Pytorch开始，到数据加载、训练、测试、调试模型，我基本掌握了通过torch框架实现一个深度学习任务的基本流程，同时对老师课上讲的梯度消失和爆炸、过拟合等问题有了更深刻的认识。